



Фаза 3.3: Архитектура Ветвящегося Дерева Состояний (State Tree / DAG Engine)

Авторы: Казбек & AI Engineering Lead (CAE Data LLC R&D)

Статус: Архитектурная спецификация (Implementation Spec)

1. Проблема: Тупики линейной истории (Linear Memory Bottleneck)

В стандартных чекпоинтерах (включая плоский PostgresSaver/SqliteSaver) история вызовов хранится как линейная лента сообщений:

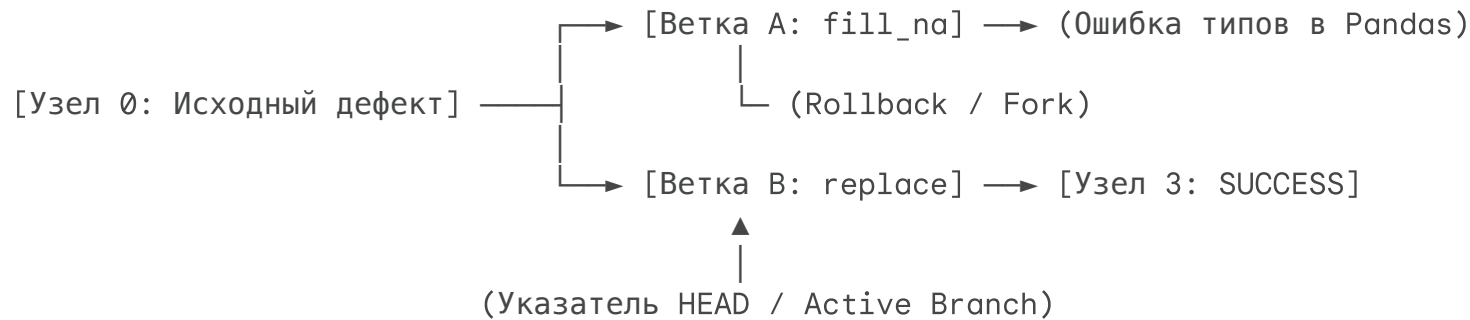
$$\text{State}_0 \longrightarrow \text{State}_1 \longrightarrow \text{State}_2 \longrightarrow \dots \longrightarrow \text{State}_N$$

Почему это приводит к системным сбоям в DataOps?

- Невозможность отката (No Rollback/Fork):** Если на Шаге 2 оператор или модель применили ошибочный патч (`replace -> "0"`), а на Шаге 4 Вычисление бонусы упало из-за искажения типов — в линейном стеке отменить шаг 2 нельзя. Вся лента сообщений уже «загрязнена» этой ошибкой.
- Потеря траекторий для DPO:** Чтобы в будущем обучать модели через Direct Preference Optimization (DPO), нам жизненно необходимы пары предсказаний:
 - Chosen (успешная ветка решения);
 - Rejected (ошибочная ветка, привешая к тупику). При плоской перезаписи истории тупиковые гипотезы просто затираются или засоряют основной промпт.

2. Решение: Git-подобное Дерево Состояний (State DAG)

В Фазе 3.3 мы переводим управление памятью на **Направленный Ациклический Граф (DAG)**. Каждое состояние сессии получает свой `node_id`, `parent_id` и относится к определенной ветке (`branch_name`).



3. Схема БД в PostgreSQL (`dataops_checkpoints`)

Для поддержки ветвления мы расширяем реляционную модель PostgreSQL двумя новыми таблицами:

Таблица `state_nodes` (Узлы графа):

- `node_id` (UUID, Primary Key) — уникальный идентификатор состояния.
- `thread_id` (Text) — сквозной идентификатор инцидента.
- `parent_id` (UUID, Nullable) — ссылка на родительский узел (Null для корня).
- `branch_name` (Text) — имя ветки (например, `main`, `hypothesis/fill_na_dept`, `hypothesis/replace_salary`).
- `node_type` (Text) — тип узла (`CHECKPOINT`, `PATCH_APPLIED`, `USER_REJECTED`, `SUPERVISOR_FORK`).
- `state_data` (JSONB) — полный снимок `AgentState`.
- `created_at` (Timestamp) — точное время фиксации.

Таблица `branch_pointers` (Указатели веток):

- `thread_id` (Text, Primary Key)
- `active_branch` (Text) — текущая ветка (по умолчанию `main`).
- `head_node_id` (UUID) — узел, на который указывает текущий `HEAD` .

4. Ключевые операции над Деревом Состояний

1. `checkpoint(state)` : Записывает новое состояние как дочерний узел текущего `HEAD` и смещает `HEAD` вперед.
2. `fork_branch(new_branch_name, target_node_id)` : Отходит от любого исторического узла `target_node_id` и создает новую независимую ветку гипотез, оставляя старую ветку нетронутой.
3. `time_travel(target_node_id)` : Смещает указатель `HEAD` на указанный узел прошлого, позволяя заново запустить граф с исторической точки.

5. Преимущество для Автономности и DPO

- **Абсолютная безопасность:** Любая критическая ошибка оператора или модели изолируется в своей ветке.
- **Сбор пар DPO:** Успешная ветка `B` объявляется `Chosen` , а тупиковая ветка `A` автоматически маркируется как `Rejected` и отправляется в датасет фоновое обучения LoRA.